

Mousiké

Relatório de Evolução - Backend MVP

Registro das mudanças realizadas em 22 de setembro de 2026

Escopo deste documento

Este relatório registra apenas o trabalho realizado hoje: correções funcionais, limpeza técnica, organização do repositório, atualização da documentação Markdown e preparação do projeto para o início do frontend. Ele não substitui a documentação técnica completa do Mousiké.

Marco atingido

BACKEND MVP PRONTO PARA FECHAMENTO

Próximo grande passo: Frontend

1. Resumo executivo

O que foi resolvido e por que esta sessão foi importante

A sessão de hoje foi dedicada ao fechamento técnico do backend do Mousiké. O foco não foi adicionar novas funcionalidades, mas corrigir pontos que poderiam causar inconsistências no frontend, remover caminhos antigos, simplificar responsabilidades e deixar o repositório mais profissional para o próximo marco do projeto.

Área	Mudança de hoje	Resultado
Publicação	Correção da primeira publicação sem Artist pré-existente	Fluxo de onboarding coerente
Uploads	Remoção do endpoint legado /uploads/audio	Menos arquivos órfãos e um único fluxo de publicação
Integração	Configuração de CORS para localhost:5173	Backend pronto para conversar com o frontend local
Desenvolvimento	Scripts npm e tsx watch	Ciclo de trabalho mais rápido e padronizado
Músicas	PATCH /songs/:id restrito ao título	audioUrl, duration e capa deixam de ser alteráveis via JSON
Configuração	Limpeza do tsconfig e centralização de env	Menos ruído e falhas de configuração mais claras
Repositório	Consolidação de .gitignore e remoção de lockfile raiz	Estrutura mais limpa
Documentação	README e database.md reescritos	Documentação alinhada ao backend atual

Situação ao final do dia

O backend está em condições de ser tratado como MVP concluído após a última validação local e o commit final. O próximo esforço de produto deve migrar para o frontend e para a integração com a API.

2. Correções funcionais realizadas hoje

2.1 Primeira publicação de música

Foi identificado um erro de lógica no fluxo de primeira publicação. Mesmo quando o usuário ainda não possuía um perfil Artist, a rota exigia o campo artists e depois tentava validar um artistId que o usuário ainda não poderia possuir.

- Agora, quando o usuário ainda não possui Artist, o backend exige artistName e pula a validação de artists.
- O backend lê os metadados do MP3, cria o Artist, cria a Song e cria SongArtist com role "main" dentro do fluxo de primeira publicação.
- Quando o Artist já existe, o campo artists continua obrigatório e a autorização exige que o Artist do usuário esteja relacionado como main.

2.2 Remoção do upload de áudio legado

O endpoint POST /uploads/audio foi removido porque permitia salvar um MP3 sem necessariamente vinculá-lo a uma música. O fluxo oficial passou a ser POST /songs/publish, que recebe os arquivos e cria a música de forma integrada.

- Removido src/routes/uploads.routes.ts.
- Removido o registro do router no server.ts.
- Removida a função createAudioUpload(), mantendo apenas a configuração de tipos de áudio usada por createSongPublishUpload().

2.3 Edição de música mais segura

A rota PATCH /songs/:id ainda aceitava duration, audioUrl e coverUrl via JSON. Isso permitia contornar as regras criadas para upload e detecção automática de duração.

- PATCH /songs/:id passa a editar somente title.
- A capa continua sendo gerenciada por rota própria de upload.
- A duração permanece derivada do arquivo MP3.
- O áudio não pode mais ser substituído apenas enviando uma URL arbitrária.
- Também foi removido o tipo auxiliar SongArtistInput que ficou sem uso.

3. Infraestrutura e experiência de desenvolvimento

3.1 CORS

O backend agora aceita requisições do frontend local em http://localhost:5173. Isso elimina o bloqueio de origem que aconteceria no navegador mesmo com as rotas funcionando normalmente por curl.

3.2 Scripts npm

O package.json foi organizado para oferecer comandos claros para cada etapa do desenvolvimento:

Comando	Objetivo
npm run dev	Executa o servidor com tsx watch e reinicia após alterações.
npm run typecheck	Executa tsc --noEmit.
npm run build	Compila o backend para dist/.
npm start	Executa a versão compilada.
npm run prisma:validate	Valida o schema Prisma.
npm run prisma:generate	Gera o Prisma Client.

3.3 TypeScript e configuração de ambiente

- Removidas opções de declaração .d.ts e .d.ts.map do tsconfig, pois o backend é uma aplicação e não uma biblioteca publicada.
- Removida a opção JSX do tsconfig, que pertence ao frontend React e não ao backend Node.
- DATABASE_URL passou a ser validada junto com JWT_SECRET em config/env.ts.

- prisma.ts deixa de confiar em process.env.DATABASE_URL! e passa a consumir a configuração validada.

4. Limpeza e organização do repositório

A revisão de hoje também separou arquivos realmente necessários de arquivos redundantes ou temporários. O objetivo é manter a raiz pequena e tornar mais fácil para qualquer pessoa entender o projeto ao abrir o GitHub.

Item	Ação	Motivo
package-lock.json na raiz	REMOVER	Não existe package.json correspondente na raiz; o lockfile válido é backend/package-lock.json.
backend/.gitignore	REMOVER	As regras ficam centralizadas no .gitignore da raiz.
.gitignore da raiz	MANTER	Deve ignorar node_modules, .env, dist, Prisma Client gerado e backend/uploads/.
backend/package-lock.json	MANTER	Registra as versões exatas das dependências do backend.
backend/prisma7.config.ts	MANTER	É parte da configuração atual do Prisma 7.
docs/database.md	MANTER	É documentação textual, fácil de revisar no GitHub e agora alinhada ao schema.
PDF/DOCX técnicos antigos	REORGANIZAR	Não executam código. Podem ser mantidos por histórico, preferencialmente em uma subpasta de documentação.

Importante sobre o GitHub atual

No momento desta revisão, o branch main ainda exibe o package-lock.json da raiz e backend/.gitignore. Isso indica que as limpezas locais feitas hoje ainda precisam entrar no commit final.

4.1 Estrutura sugerida para docs/

Para deixar o repositório visualmente mais organizado sem perder histórico, a documentação binária pode ser agrupada por finalidade:

```
docs/
├── database.md
├── technical/
│   ├── Mousiké - Documentação Técnica do Projeto.docx
│   └── Mousiké - Documentação Técnica do Projeto.pdf
├── changelog/
│   └── 2026-09-22-backend-mvp/
│       ├── Mousike_Relatorio_Evolucao_2026-09-22.docx
│       └── Mousike_Relatorio_Evolucao_2026-09-22.pdf
```

5. Documentação atualizada hoje

5.1 README.md

O README foi reescrito por completo para representar o projeto como ele existe agora, e não como existia nas primeiras etapas. Ele passa a apresentar o Backend MVP, as tecnologias atuais, as funcionalidades implementadas, a estrutura do repositório, configuração de ambiente, scripts, uploads, segurança e o roadmap do frontend.

5.2 docs/database.md

O documento do banco também foi reescrito para refletir o schema Prisma atual. Agora ele documenta as entidades, tabelas de relacionamento, chaves compostas, índices, onDelete, regras de domínio e decisões de modelagem.

- User e Artist 1:0..1.
- Song-Artist, Song-Genre, Album-Artist e Playlist-Song N:N.
- Favorite único por usuário e música.
- ListeningHistory com índices para usuário/data e música.
- Follow de artistas e follow entre usuários.

6. O que o Mousiké já possui após este marco

O Backend MVP reúne um conjunto de funcionalidades suficiente para iniciar a construção do cliente frontend sem precisar redesenhar a base da API.

Domínio	Estado atual
Autenticação	Cadastro, login, JWT, /auth/me e middleware de autenticação.
Usuários	Perfil, edição, senha, bio, imagem e exclusão protegida.
Artistas	Perfil integrado ao User, imagem, bio, ownership e criação automática na primeira publicação.
Músicas	Publicação de MP3, leitura de duração, capa, múltiplos artistas, gêneros e autorização por main artist.
Álbuns	Criação, artistas, músicas, data de lançamento, capa e regras de propriedade.
Playlists	Públicas/privadas, capa, músicas, posição, ownership e ordenação.
Favoritos	Relação única usuário-música e listagem por usuário.
Histórico	Registro após 25% reproduzido e armazenamento de segundos ouvidos.
Seguidores	Follow de artistas e de outros usuários, com proteção contra auto-follow.
Busca	Busca global de músicas, artistas e álbuns.
Recomendações	Personalização por histórico, favoritos, artistas seguidos, recência, gêneros e diversidade.

Home/Mixes	Home personalizada, mix for-you, discover e favorites.
Uploads	Imagens e MP3 com limpeza de arquivos substituídos ou órfãos.

7. Estrutura técnica consolidada

A arquitetura continua propositalmente simples para o tamanho do projeto. Não foi introduzida uma camada ampla de controllers/repositories apenas por padrão. Services são usados onde a lógica realmente justificou separação, como no sistema de recomendações.

```

backend/src/
├─ config/      -> ambiente e upload
├─ lib/         -> Prisma, JWT e arquivos
├─ middlewares/ -> autenticação e erros de upload
├─ routes/      -> endpoints por domínio
├─ services/    -> recomendações
├─ types/       -> extensões de tipos
└─ server.ts    -> composição da aplicação

```

8. Validação antes do commit final

Antes de publicar o fechamento do backend, o fluxo recomendado é executar novamente a validação completa e confirmar a árvore de arquivos:

```
cd C:\Mousike\backend
```

```
npm run typecheck
```

```
npm run build
```

```
cd C:\Mousike
```

```
git status
```

No git status, a expectativa é visualizar as alterações de código/documentação de hoje e as exclusões planejadas do package-lock.json da raiz e de backend/.gitignore. Arquivos .env, uploads, dist, node_modules e Prisma Client gerado não devem aparecer para commit.

9. Próximo passo: iniciar o frontend

Com o backend estabilizado, o próximo marco deixa de ser infraestrutura de API e passa a ser experiência de produto. A recomendação é criar o frontend e conectá-lo progressivamente às rotas existentes, começando por autenticação e navegação base.

1. Criar a estrutura do frontend (provavelmente Vite + React + TypeScript).
2. Definir a identidade visual e o layout base do Mousiké.
3. Criar cadastro e login.
4. Armazenar e enviar o JWT nas requisições autenticadas.
5. Integrar /auth/me e a home personalizada.
6. Construir busca, cards de música/artista/álbum e páginas de detalhe.
7. Criar o player e a lógica de progresso da reprodução.
8. Integrar o registro de ListeningHistory quando o usuário atingir 25%.

9. Construir playlists, favoritos e follows.
10. Criar o fluxo de publicação de músicas e edição de perfil Artist.

Primeiro objetivo do frontend

Conseguir abrir o app, cadastrar/logar um usuário, recuperar /auth/me e renderizar a home usando dados reais do backend. Depois disso, avançar para player, busca e bibliotecas.

10. Melhorias pós-MVP que podem esperar

A revisão final também identificou melhorias válidas, mas que não precisam bloquear o frontend. Elas podem entrar em uma etapa de robustez ou preparação para deploy.

- Controle administrativo para criação/edição de gêneros.
- Modelo mais explícito de permissões em álbuns colaborativos.
- Rate limiting e headers de segurança.
- Validação estruturada de payloads com uma biblioteca dedicada.
- Paginação em listagens maiores.
- Refresh tokens ou estratégia de revogação de sessão.
- Testes automatizados de integração e unidade.
- Observabilidade, logging estruturado e tratamento global de erros.

11. Checklist de fechamento do Backend MVP

Status	Item
CONCLUÍDO	Correção da primeira publicação
CONCLUÍDO	Remoção de /uploads/audio
CONCLUÍDO	CORS para o frontend local
CONCLUÍDO	Scripts npm e tsx
CONCLUÍDO	PATCH /songs/:id restrito ao título
CONCLUÍDO	tsconfig limpo
CONCLUÍDO	DATABASE_URL centralizada em env.ts
CONCLUÍDO	README reescrito
CONCLUÍDO	database.md reescrito
ANTES DO COMMIT	Remover lockfile da raiz no commit final
ANTES DO COMMIT	Remover backend/.gitignore no commit final
ANTES DO COMMIT	Executar typecheck + build finais
ANTES DO COMMIT	Commit/push de fechamento do backend

Conclusão

O trabalho de hoje transformou o backend de um conjunto funcional de rotas em um MVP mais coerente, previsível e pronto para integração. O maior ganho foi reduzir caminhos alternativos e inconsistências antes que o frontend começasse a depender deles.